

动态1

C++

```
1  #include<iostream>
2  using namespace std;
3
4  //Point类
5  class Point
6  {
7      private:int x,y;
8      public:
9
10     //构造函数，输出 cout<<"\nPoint is called!"; 并完成私有成员的初始化
11     Point(int xx,int yy):x(xx),y(yy){cout<<"\nPoint is called!";};
12     Point(int xx):x(xx),y(0){cout<<"\nPoint is called!";};
13     Point():x(0),y(0){cout<<"\nPoint is called!";};
14     //Point(){cout<<"\nPoint is called!";}
15
16     //析构函数，输出    cout<<"\n~Point is called!";
17     ~Point()
18     {
19         //delete this; 析构函数会自动调用并释放对象的资源 ， 所以不用加这句话
20         cout<<"\n~Point is called!";
21     };
22
23     //友元输出函数，输出    "("<<p.x<<","<<p.y<<")";
24     //加<T>, 因为友元函数的定义在模板类的外部
25     friend ostream& operator<< (ostream& out,const Point& p);
26 };
27
28 ostream& operator<< (ostream & out,const Point& p)
29 {
30     out<<"("<<p.x<<","<<p.y<<")";
31     return out;
32 }
33
34 //DynamicArray类
35 template <typename T>
36 //模板函数 - 各种数据类型都能使用
37 class DynamicArray
38 {
39     private:
40         //私有成员，外部想使用必须通过set接口
41         T* array; //pointer ， 一个T类型的指针
42         unsigned int mallocSize; //分配空间的大小。
43
44     public:
45         //Constructors
46         // cout<<endl<< "new T["<<this->mallocSize<<"] malloc "<< this->mallocSize << "*"<<sizeof(T)<<"]="<<this->m
mallocSize *sizeof(T)<<" bytes memory in heap";
47         //构造函数 - 分配空间数+每个空间的默认大小（动态数组有多长+每个数是什么）
48         //content为只读对象，可以放常值或者a+b这种
49         DynamicArray(unsigned length, const T &content); // mallocSize=length; 设置每个元素的初始内容是 content;
50
51         //Copy Constructor
52         // cout<<endl<< "Copy Constructor is called";
53         //copy构造函数
54         DynamicArray(const DynamicArray<T> & anotherDA ) ;
55
56         //无参数构造函数
57         //和上一题比起来只添加了一个无参数的构造函数
58         DynamicArray():array(NULL),mallocSize(0){};
```

```

58
59     // Destructors
60     // cout<<endl<< "delete[] array free "<< this->mallocSize << "*"<<sizeof(T)<< "="<<this->mallocSize *sizeof
61 (T)<<" bytes memory in heap";
62     //析构函数 - 在对象被销毁时自动调用，用于清理对象所占用的资源
63     ~DynamicArray();
64
65     //return the this->mallocSize
66     //返回动态数组的大小
67     unsigned int capacity() const;
68
69     // for the array[i]=someT.
70     //重载[]操作符，用于访问数组元素 - 成员函数，可以访问私有成员
71     T& operator[](unsigned int i) ;
72
73     //自己定义个operator[] const 重载
74     //const的[]操作符重载
75     T& operator[] (unsigned int i) const;
76
77     //自己定义个 operator = 重载
78     // 函数内要输出 cout<<endl<<"operator = is called";
79     //重载=操作符
80     DynamicArray<T> & operator= (const DynamicArray<T> & anotherDA ) ;
81
82     //数组空间动态调整函数
83     int resize(unsigned int newSize, const T& ValOfNewItems) ;
84 };
85
86
87 //深拷贝 - 构造函数
88 template <typename T>
89 DynamicArray<T>::DynamicArray(const DynamicArray<T> &a):mallocSize(a.mallocSize)
90 {
91     cout<<endl<< "Copy Constructor is called";
92     //先得到长度，再申请一个新的空间
93     mallocSize=a.mallocSize;
94     array=new T[mallocSize];
95
96     //把数组元素一个一个复制过来
97     //仅仅是数值，可以不用写，直接用程序送的浅拷贝
98     //涉及到对象申请，都要自己写拷贝构造函数-深拷贝
99     for(int i=0;i<mallocSize;i++)
100     {
101         array[i]=a.array[i];
102     }
103 }
104
105 //构造函数
106 //构造函数要写在模板外边 - array[1]的每个值都是c
107 //new可能因为内存不够不成功 - 或者因为没有连续的空间无法申请那么多空间
108 template <typename T>
109 DynamicArray<T>::DynamicArray(unsigned length, const T &content):mallocSize(length)
110 {
111     //在构造函数中添加相应的输出语句
112     //sizeof(T)用于获取类型“T”的大小
113     //可以直接用length
114     cout<<endl<< "new T["<<length<<"] malloc "<<length<< "*"<<sizeof(T)<< "="<<length*sizeof(T)<< " bytes memory in
115 heap";
116     array=new T[length];
117     mallocSize=length;
118     for(unsigned int i=0;i<length;i++)
119     {
120         array[i]=content;
121     }
122     //cout的位置不一样
123
124     //cout<<"new T["<<len<<"] malloc "<<len<<"*"<<sizeof(T)<< "="<<len*sizeof(T)<< " bytes memory in heap"<<endl;

```

```

125 //析构函数 - 归还资源
126 template <typename T>
127 DynamicArray<T>::~~DynamicArray()
128 {
129     //在析构函数中添加相应的输出语句
130     //sizeof(T)用于获取类型“T”的大小
131     //只能用this→mallocSize
132     cout<<endl<< "delete[] array free "<<mallocSize << "*"<<sizeof(T)<<"="<<mallocSize *sizeof(T)<<" bytes memor
133 y in heap";
134     if(this≠NULL)
135     {
136         //没有[]会析构1个
137         //有[]相当于告诉它是数组，所以会析构所有的
138         delete[] array;
139     }
140     array=NULL;
141     //cout的位置不一样
142     //cout<<"delete[] array free "<<this→mallocSize<<"*"<<sizeof(T)<<"="<<this→mallocSize*sizeof(T)<<" bytes m
143 emory in heap"<<endl;
144 }

145 //返回容量值
146 template <typename T>
147 unsigned int DynamicArray<T>::capacity() const
148 {
149     return mallocSize;
150 }
151
152 //operator[]重载
153 //用于实现: iarray[0]=999
154 //否则得: iarray.array[0]=999
155 //操作符重载
156 template <typename T>
157 T& DynamicArray<T>::operator[](unsigned int i)
158 {
159     //越界会提示
160     if(i<mallocSize&&i≥0)
161     {
162         return array[i];
163     }
164     else
165     {
166         cout<<"out of range";
167     }
168 }
169
170 //operator[]const重载
171 template <typename T>
172 T& DynamicArray<T>::operator[](unsigned int i) const
173 {
174     if(i<mallocSize&&i≥0)
175     {
176         return array[i];
177     }
178     else
179     {
180         cout<<"out of range";
181     }
182 }
183
184 //深赋值 - operator=重载
185 template <typename T>
186 DynamicArray<T> & DynamicArray<T>::operator= (const DynamicArray<T> & a )
187 {
188     cout<<endl<<"operator = is called";
189     if(this==&a)
190     {
191         return *this;
192     }

```

```

193 //没有[]会析构1个
194 //有[]相当于告诉它是数组，所以会析构所有的
195 delete[] array;
196 //先得到长度，再申请一个新的空间
197 mallocSize=a.mallocSize;
198 array=new T[mallocSize];
199
200 for(int i=0;i<mallocSize;i++)
201 {
202     array[i]=a.array[i];
203 }
204
205 //在函数末尾return *this - 为了连续赋值
206 return *this;
207 }
208
209 //数组空间动态调整函数
210 template <typename T>
211 int DynamicArray<T>::resize(unsigned int newSize, const T& ValOfNewItems)
212 {
213     cout<<"\nresize is called";
214     if(newSize>mallocSize)
215     {
216         //分配新空间
217         T* newarray=new T[newSize];
218         //先复制原来的数据 - 不能用size
219         for(unsigned int i=0;i<mallocSize;i++)
220         {
221             newarray[i]=array[i];
222         }
223         //再对新数据赋值 - 不能用size
224         for(unsigned int i=mallocSize;i<newSize;i++)
225         {
226             newarray[i]=ValOfNewItems;
227         }
228         //释放原来的空间
229         delete[] array;
230         //更新
231         array=newarray;
232         mallocSize=newSize;
233         return 1;
234     }
235     else if(newSize==mallocSize)
236     {
237         return 0;
238     }
239     else if(newSize<mallocSize)
240     {
241         //分配新空间
242         T* newarray=new T[newSize];
243         //仅复制原来的数据
244         for(unsigned int i=0;i<newSize;i++)
245         {
246             newarray[i]=array[i];
247         }
248         //释放原来的空间
249         delete[] array;
250         //更新
251         array=newarray;
252         mallocSize=newSize;
253         return -1;
254     }
255 }
256
257 //StudybarCommentBegin
258 int main()
259 {
260     int i,j;
261     DynamicArray<int> a(20,0);
262     DynamicArray<DynamicArray<int> > b(10,a);

```

```
263     b[0].resize(30,1);
264     b[5].resize(10,5);
265     for(i=0;i<10;i++)
266     {   cout<<"\n";
267         for(j=0;j<b[i].capacity();j++)
268         {   cout<<" "<<b[i][j] ;}
269     }
270     return 0;
271 }
272 //StudybarCommentEnd
273
274
```